

做长序列，保高速度：抖音十亿规模推荐中的端到端 10k 序列建模

Lin Guan*

guanlin.13@gmail.com

ByteDance
北京, 中国

Jia-Qi Yang*

yangjiaqi.yjq@bytedance.com

ByteDance
上海, 中国

Zhishan Zhao*

zhaozhishan@bytedance.com

ByteDance
北京, 中国

Beichuan Zhang*

zhangbeichuan.123@bytedance.com

ByteDance
北京, 中国

Bo Sun

zhangyang.2583@bytedance.com

ByteDance
圣何塞, CA, 美国

Xuanyuan Luo

xuanyuanluo@bytedance.com

ByteDance
杭州, 浙江, 中国

Jinan Ni

nijinan@bytedance.com

ByteDance
上海, 中国

Xiaowen Li

lixiaowen.911@bytedance.com

ByteDance
北京, 中国

Yuhang Qi

qiyuhang@bytedance.com

ByteDance
杭州, 浙江, 中国

Zhifang Fan

fanzhifangfzf@gmail.com

ByteDance
上海, 中国

Hangyu Wang

wanghangyu.123@bytedance.com

ByteDance
上海, 中国

Qiwei Chen[†]

chenqiwei05@gmail.com

ByteDance
上海, 中国

Yi Cheng

chengyi.23@bytedance.com

ByteDance
北京, 中国

Feng Zhang

feng.zhang@bytedance.com

ByteDance
上海, 中国

Xiao Yang

wuqi.shaw@bytedance.com

ByteDance
北京, 中国

摘要

抖音等短视频推荐系统必须在不突破延迟和成本预算的前提下，充分利用极长的用户历史。本文提出一个端到端系统，将生产环境中的长序列建模扩展至长度为 $10k$ 的历史。首先，我们提出堆叠式目标到历史交叉注意力 (*Stacked Target-to-History Cross Attention, STCA*)，以从目标到历史的堆叠交叉注意力替代历史

自注意力，将复杂度随序列长度的增长从二次降至线性，并支持高效的端到端训练。其次，我们提出请求级批处理 (*Request Level Batching, RLB*)，这是一种以用户为中心的批处理方案，将同一用户/请求的多个目标聚合起来，共享用户侧编码，从而在不改变学习目标的情况下显著降低与序列相关的存储、通信和计算开销。第三，我们设计了一种长度外推训练策略——在较短窗口上训练，在长得多的窗口上推理——使模型无需额外训练成本即可泛化到 $10k$ 长度的历史。在离线和在线实验中，随着历史长度和模型容量的扩展，我们观察到可预测的单调增益，与大语言模型中观察到的缩放定律行为相呼应。该系统已在抖音全流量部署，在满足生产延迟要求的同时显著改善了关键参与度指标，展示了将端到端长序列推荐扩展至 $10k$ 规模的一条实用途径。

*贡献相同。

[†]通讯作者。



This work is licensed under a Creative Commons Attribution 4.0 International License.

WWW '26, 阿拉伯联合酋长国迪拜

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2307-0/2026/04

<https://doi.org/10.1145/3774904.3792811>

CCS 概念

• 信息系统 → 推荐系统.

Keywords

推荐系统, 长序列建模, 缩放定律

ACM 参考文献格式:

Lin Guan, Jia-Qi Yang, Zhishan Zhao, Beichuan Zhang, Bo Sun, Xuanyuan Luo, Jinan Ni, Xiaowen Li, Yuhang Qi, Zhi-fang Fan, Hangyu Wang, Qiwei Chen, Yi Cheng, Feng Zhang, and Xiao Yang. 2026. 做长序列, 保高速度: 抖音十亿规模推荐中的端到端 10k 序列建模. In *2026 年 ACM Web 大会论文集 (WWW '26)*, 2026 年 4 月 13–17 日, 阿拉伯联合酋长国迪拜. ACM, New York, NY, USA, 14 pages. <https://doi.org/10.1145/3774904.3792811>

1 引言

深度神经网络已成为现代推荐系统的支柱, 为电子商务、信息流和短视频平台等应用提供支持 [4, 5, 8, 38]。其中一个关键原因是, 它们能够利用用户行为序列, 因为过去的交互为推断用户偏好提供了重要信号 [10, 16]。在抖音等短视频推荐场景中 [41], 用户历史通常长达数千个视频。如果能得到有效利用, 这些长序列可以显著提升排序性能 [13, 33, 40]。

对更长序列进行建模的重要性与深度学习的扩展定律有关: 随着数据量、参数量和计算量的增加, 性能往往会以可预测的方式提升 [11, 14]。不同于 NLP/CV 通常通过扩大数据集实现扩展, 推荐系统受到用户生成数据的约束 [20, 30]。让模型接触更多信息的一种自然方式是使用更长的历史序列 [3, 9, 17]。然而, 大多数系统采用两阶段范式 [2, 25, 27, 31]: 检索一小组与目标相似的条目, 并将截断后的序列输入排序器。这种方式虽然高效, 却破坏了端到端优化, 并丢弃了有价值的交互。我们的实证结果 (图 1) 表明, 当架构和系统支持真正的端到端长序列训练与推理时, 模型质量会随序列长度和序列模块容量平稳扩展, 与其他模式中的扩展定律行为相呼应。

要真正释放大规模推荐模型的扩展潜力, 长序列上的端到端训练必须与严格的在线延迟和成本预算共

存 [15, 35]。设计必须有选择地分配计算, 而更长的历史序列会放大分布式训练中的存储、通信和计算开销 [19, 21, 22]。为此, 我们结合了: (i) 一种以目标为中心的单查询交叉注意力模型 (STCA), 其每层成本与序列长度呈线性关系; (ii) 请求级批处理 (*Request Level Batching*, *RLB*), 它在一次请求的多个目标之间摊销用户侧编码, 并且可扩展为在同一用户/会话的多个请求之间共享, 从而进一步提高效率; 以及 (iii) 一套“稀疏训练/稠密推理”方案: 我们在平均长度约为 $2k$ 个 token 的序列上训练, 却在服务时外推至 $10k$, 在不增加训练计算量的情况下保留端到端建模。综合这些组件, 随着长度和容量增加, 模型能够获得可预测的增益, 符合扩展定律行为 (图 1)。

我们的贡献。我们针对实用的端到端长序列推荐做出了三项贡献:

- **序列高效架构。**我们提出堆叠式目标到历史交叉注意力 (STCA), 优先建模目标条目与历史之间的交叉注意力, 同时省略历史自注意力, 从而将关于序列长度 L 的复杂度从 $O(L^2)$ 降至 $O(L)$ 。堆叠多层通过目标条件融合捕获高阶依赖关系。
- **通过以用户为中心的批处理实现可扩展训练。**我们提出请求级批处理 (*RLB*), 聚合来自同一用户的样本, 使多个目标共享一次用户侧编码。除单次请求外, *RLB* 还能自然扩展为同一用户/会话的多请求共享, 在保持经验风险无偏估计的同时, 进一步降低内存、通信和计算开销 (据报告, 请求级共享下最多可减少 $8\times$)。
- **稀疏训练, 稠密推理。**我们采用长度外推方案, 在平均长度为 $\sim 2k$ 的序列上训练、在最长 $10k$ 的序列上提供服务, 从而将训练成本与部署时的上下文长度解耦, 并在不增加训练计算量的情况下实现长序列增益。

这些创新为在生产约束下沿序列维度扩展提供了实用框架。部署于某大型短视频平台后, 我们的方法在多项业务指标上取得了显著的在线增益。

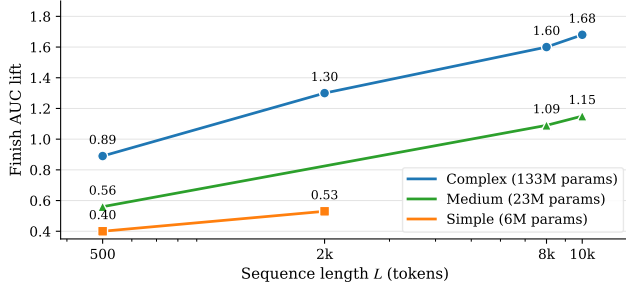


图 1: 随序列长度和模型容量的扩展。随着用户序列长度(500 → 10k 个 token)和序列模块容量(Simple: 6M、Medium: 23M、Complex: 133M 参数)的增加, 序列模块 (STCA) 的 Finish AUC 提升 (百分点)。

2 背景与符号说明

符号说明。全文使用如下符号。用户交互历史为长度为 L 的 $\mathcal{H} = \{(v_i, a_i)\}_{i=1}^L$, 其中 v_i 是第 i 个历史视频的特征向量, a_i 是其交互类型; t 表示待排序的候选 (目标) 视频。 (v_i, a_i) 的嵌入为 $\mathbf{x}_i$, t 的嵌入为 $\mathbf{x}_t$ 。令 d 为嵌入维度, r 为 *SwiGLUFFN* 宽度中的扩展比, h 为注意力头数, M 为堆叠交叉注意力层数。模型输出 $\hat{y} \in [0, 1]$, 即 t 的预测完播概率, $y \in \{0, 1\}$ 为真实完播标签。我们将每次历史交互 (v_i, a_i) 称为一个 *token*; 因此, 序列长度 L 表示历史交互 (token) 的数量。

任务。我们研究大规模短视频推荐 (例如抖音/TikTok), 其中系统为用户对一组候选视频进行排序。在实践中, 最终得分可能结合多个目标 (完播、点击等); 为清晰起见, 我们重点预测完播率。给定输入 $(\mathcal{H}, t)$, 排序模型输出标量 $\hat{y} \in [0, 1]$, 即在 $\mathcal{H}$ 条件下用户看完 t 的估计概率。在介绍模型和训练策略时, 全文均使用这套符号。

3 方法

如图 2 所示, 我们为短视频排序中的长序列建模构建了一个端到端框架, 将序列高效架构、以用户为中心的批处理策略和长度外推训练方案结合起来。

3.1 堆叠式目标交叉注意力 (STCA)

在排序中, 预测用户对 t 的响应时, 主要信号来自 t 与用户历史行为之间的直接交互, 而历史条目之间的二阶关系相对而言信息量较少。然而, 在 $[t; \mathcal{H}]$ 上使用 Transformer 式自注意力会产生关于历史长度的 $O(L^2)$ 开销, 因此限制了 L 。

我们明确地在容量与成本之间进行权衡: 弱化显式的历史—历史交互, 转而使用单查询的目标到历史交叉注意力 (STCA)。通过将目标作为唯一查询, 每层复杂度关于 L 变为线性 ($O(Ldh)$), 使计算精确聚焦于目标与历史的相关性。此外, 在单查询条件下, 我们可以重排计算 (第 A.2.2 节), 避免实例化长度为 L 的投影键/值张量 (例如 XW_K 和 XW_V), 进一步减少内存和随长度变化的 FLOPs。这种大幅度的 FLOPs 和内存削减, 使得在相同计算预算内用超长历史 (例如 10k 级别) 进行训练和服务成为可能, 从而改善纵向扩展能力: 在计算量相当时, STCA 可以处理长得多的上下文, 并且比必须在较短 L 上运行的 $O(L^2)$ 自注意力取得更高准确率。

3.1.1 输入编码。每个历史元素 (v_j, a_j) 被嵌入为 $\mathbf{x}_j \in \mathbb{R}^d$ (融合视频、动作类型和位置) 以及 $X = [\mathbf{x}_1, \dots, \mathbf{x}_L] \in \mathbb{R}^{L \times d}$ 。目标视频被嵌入为 $\mathbf{x}_t \in \mathbb{R}^d$ 。我们使用一个维度保持不变的 *SwiGLUFFN* 模块 (SwiGLU + 线性投影), 随后接 LayerNorm:

$$\text{SwiGLUFFN}(\mathbf{x}) = \left((\mathbf{x}W_u) \odot (\mathbf{x}W_v \odot \text{sigmoid}(\mathbf{x}W_v)) \right) W_o, \quad (1)$$

其中, $W_u, W_v \in \mathbb{R}^{d \times rd}$, $W_o \in \mathbb{R}^{rd \times d}$, $r \geq 1$, 而 $\odot$ 表示逐元素乘积 (省略偏置项)。我们将该模块逐行应用于矩阵, 随后进行归一化:

$$\tilde{X}^{(i)} = \text{LN}(\text{SwiGLUFFN}^{(i)}(X)) \in \mathbb{R}^{L \times d}, \quad (2)$$

$$\mathbf{q}^{(1)} = \text{LN}(\text{SwiGLUFFN}^{(1)}(\mathbf{x}_t)) \in \mathbb{R}^d, \quad (3)$$

其中, $\text{LN}(\cdot)$ 表示 LayerNorm。

多头目标到历史交叉注意力。在第 i 层, 给定 $\mathbf{q}^{(i)}$ 和 $\tilde{X}^{(i)}$, 我们计算 h 头交叉注意力。令 $d_h = d/h$,

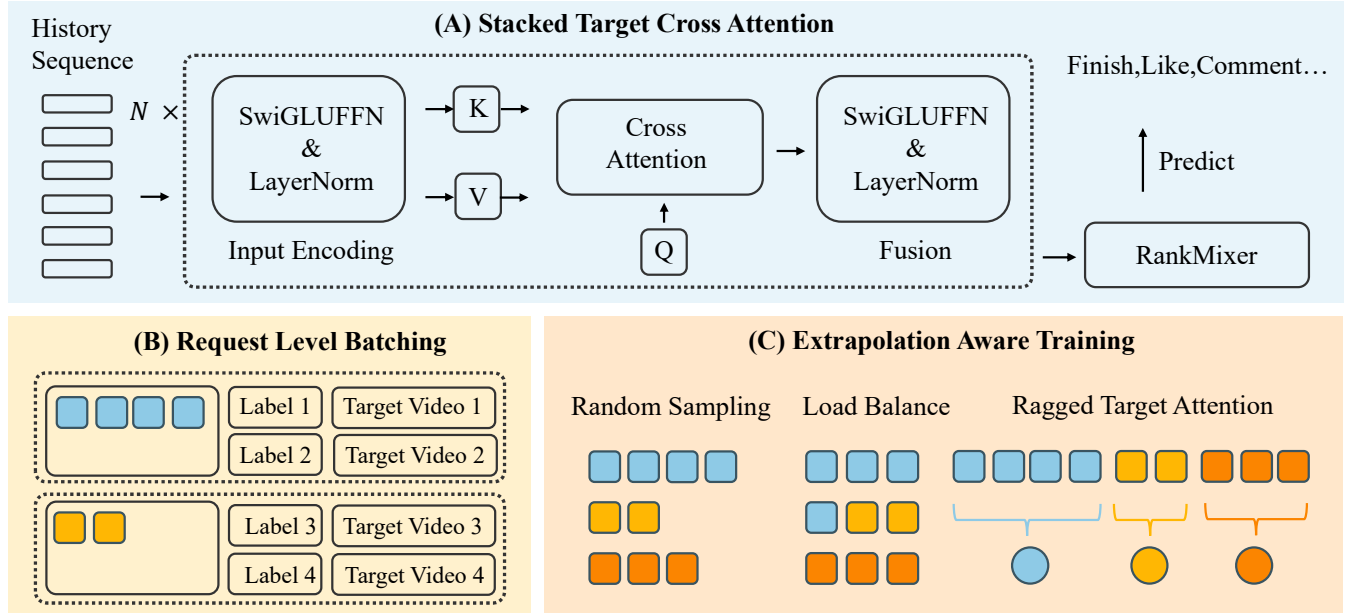


图 2: 长历史排序系统栈概览。(A) 堆叠式目标交叉注意力: 从目标到完整历史的单查询交叉注意力, 并进行逐层融合, 使历史长度能够线性扩展并支持端到端优化。(B) 请求级批处理: 每个请求仅计算一次用户/历史编码, 并在多个目标之间复用, 以降低带宽和计算开销。(C) 外推感知训练: 训练时采样较短历史, 推理时使用较长历史 (稀疏训练, 稠密推理)。

并且

$$W_Q^{(i,j)}, W_K^{(i,j)}, W_V^{(i,j)} \in \mathbb{R}^{d \times d_h}, \quad W_O^{(i)} \in \mathbb{R}^{d \times d}.$$

对于第 $j \in \{1, \dots, h\}$ 个头,

$$\alpha^{(i,j)} = \text{softmax} \left(\frac{\mathbf{q}^{(i)} W_Q^{(i,j)} (\tilde{X}^{(i)} W_K^{(i,j)})^\top}{\sqrt{d_h}} \right) \in \mathbb{R}^{1 \times L}, \quad (4)$$

$$\mathbf{o}^{(i,j)} = \alpha^{(i,j)} (\tilde{X}^{(i)} W_V^{(i,j)}) \in \mathbb{R}^{1 \times d_h}. \quad (5)$$

拼接各个头并进行投影:

$$\mathbf{o}^{(i)} = [\mathbf{o}^{(i,1)} \parallel \dots \parallel \mathbf{o}^{(i,h)}] W_O^{(i)} \in \mathbb{R}^d. \quad (6)$$

每层针对单个目标查询的成本为 $O(Ldh)$ ——关于 L 呈线性——而在 $[t; \mathcal{H}]$ 上进行自注意力的成本为 $O(L^2 dh)$ 。

3.1.2 堆叠与目标条件融合。 我们堆叠 M 个交叉注意力层, 并通过目标条件融合更新查询。为保持维度一致 (输入为 d), 我们使用可学习投影压缩不断增长

的拼接结果:

$$\mathbf{q}^{(i+1)} = \text{LN} \left(\text{SwiGLUFFN}^{(i+1)} \left([\mathbf{o}^{(1)} \parallel \dots \parallel \mathbf{o}^{(i)} \parallel \mathbf{x}_t] W_C^{(i+1)} \right) \right), \quad (7)$$

其中, $W_C^{(i+1)} \in \mathbb{R}^{(i+1)d \times d}$ 。经过 M 层后, 我们得到逐层摘要 $\{\mathbf{o}^{(i)}\}_{i=1}^M$, 随后将其与目标嵌入结合用于预测。

3.1.3 预测头与目标函数。 我们将所有层的摘要与目标一起压缩, 形成最终的目标感知 token:

$$\mathbf{z} = \text{SwiGLUFFN} \left([\mathbf{o}^{(1)} \parallel \dots \parallel \mathbf{o}^{(M)} \parallel \mathbf{x}_t] W_Z \right), \quad W_Z \in \mathbb{R}^{(M+1)d \times d} \quad (8)$$

$$\mathcal{X}_{\text{mix}} = \text{concat}(\mathbf{z}, \{\mathbf{u}_k\}_{k=1}^K, \{\mathbf{c}_\ell\}_{\ell=1}^C),$$

其中, $\{\mathbf{u}_k\}$ 是辅助的用户侧 token (例如用户画像/上下文特征), $\{\mathbf{c}_\ell\}$ 是与同一目标 t 关联的候选侧 token

(例如内容/创作者模态)。随后, RankMixer[41] 产生

$$\mathbf{h} = \text{RankMixer}(\mathcal{X}_{\text{mix}}; \Theta), \quad \hat{y} = \text{sigmoid}(\mathbf{w}^\top \mathbf{h} + b). \quad (9)$$

我们优化二元交叉熵:

$$\mathcal{L}_{\text{BCE}} = -y \log \hat{y} - (1-y) \log(1-\hat{y}). \quad (10)$$

3.1.4 单查询交叉注意力的计算优化。 当每层恰好只有一个查询时, 我们可以重排计算, 从而避免实例化长度为 L 的投影键/值张量 (例如 XW_K 和 XW_V)。具体而言, 对于 $X \in \mathbb{R}^{L \times d}$ 和 $q \in \mathbb{R}^{1 \times d}$, 单查询交叉注意力可实现为

$$\text{Attn}(q, X) = \left(\underbrace{\text{softmax}\left(\frac{(qW_Q)W_K^\top X^\top}{\sqrt{d_h}}\right)}_{\alpha \in \mathbb{R}^{1 \times L}} X \right) W_V = (\alpha X) W_V. \quad (11)$$

该形式避免了任何 $L \times d_h$ 中间量, 并降低了与长度相关的 FLOPs。完整推导与 FLOPs 分析见附录 A.2.2。

3.2 请求级批处理 (RLB)

STCA 使每个目标的序列成本随历史长度 ($O(L)$) 线性增长, 从而支持长上下文。然而, 在真实日志中, 每个用户通常会在同一请求/会话内产生多个目标。如果仍然基于独立三元组 (u, t, y) 训练, 则同一长历史 $\mathcal{H}$ 会被反复序列化、传输 (CPU→GPU) 和重新编码; 随着 L 增大, 存储与带宽而非 FLOPs 将成为瓶颈。因此, RLB 是 STCA 在系统侧的补充: 它消除了冗余的用户侧工作, 使关于 L 线性的编码器能够在生产预算内扩展至数千长度的序列。

3.2.1 基本思想与无偏性。 RLB 将同一用户的 m 个样本聚合为一个用户微批次 $\mathcal{B}_u = \{(u, t_k, y_k)\}_{k=1}^m$ 。令 $\Phi_{\text{user}}(\mathcal{H})$ 表示用户/历史路径 (在各目标间共享)。采用 RLB 后, 我们只计算一次 $\Phi_{\text{user}}(\mathcal{H})$, 并将其复用至全部 $\{t_k\}_{k=1}^m$ 。每用户损失与总体目标为

$$\mathcal{L}_u = \frac{1}{m} \sum_{k=1}^m \mathcal{L}_{\text{BCE}}(\hat{y}(u, t_k), y_k), \quad \mathcal{L} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \mathcal{L}_u. \quad (12)$$

无偏性。 将传统目标写成用户平均下的实例平均, 可以看出: 用对 m 个目标进行的不放回抽样平均替代

内部平均, 并不会改变期望 (由期望的线性性可知)。因此, 式 (12) 是经验风险的无偏估计量; RLB 仅改变计算布局, 而不改变学习目标。请求内目标之间的相关性不会影响这种无偏性, 因为 RLB 只对样本重新分组, 并未改变损失定义。

3.2.2 系统视角。 RLB 将重复的用户侧工作转化为计算一次、复用 m 次的模式: 我们在每个请求中只传输并编码一次长历史 $\mathcal{H}$, 并在多个目标间复用共享的用户/历史表示。这样可减少冗余的主机 ↔ 设备传输与激活复制, 通过批处理目标侧计算提高核函数利用率, 并通过减少每步不同用户编码的数量来降低分布式开销。端到端测量显示, 启用 RLB 后带宽降低了 **77–84%** (详见第 4.3 节)。

3.3 外推: 稀疏训练, 稠密推理

STCA 使每个目标的序列成本关于 L 线性增长, 而 RLB 将用户路径分摊到 m 个目标上, 二者共同支持在严格延迟和带宽预算下进行长上下文服务。然而, 在均匀的长历史上训练时, 成本仍会随每个批次处理的 token 数量线性增长, 并随着 L 增大而迅速耗尽内存与吞吐能力。为将训练成本与部署时的上下文长度解耦, 我们引入一种长度外推方案: 稀疏训练 (每批次平均 token 数较少), 但稠密推理 (测试时使用长历史)。本节始终将部署目标固定为 $L_{\text{infer}} = 10\text{k}$, 将训练平均长度固定为 $L_{\text{train}}^{\text{avg}} = 2\text{k}$, 得到外推比率 $\rho_{\text{extra}} = \frac{L_{\text{infer}}}{L_{\text{train}}^{\text{avg}}} = 5$ 。

因此, 我们在训练中采用随机长度 (Stochastic Length, SL) 训练范式: 将每个输入序列随机截断为长度 $L_{\text{train}} \in [L_{\text{train}}^{\min}, L_{\text{train}}^{\max}]$, 其中 $L_{\text{train}}^{\max} \leq L_{\text{infer}}$, 而 L_{infer} 是推理时采用的最大序列长度。我们通过序列稀疏度 (sequence sparsity, SS) 量化由此产生的效率, 其定义为

$$\text{SS} = \frac{\mathbb{E}[L_{\text{train}}]}{L_{\text{train}}^{\max}} = \frac{L_{\text{train}}^{\text{avg}}}{L_{\text{train}}^{\max}}, \quad (13)$$

它反映了相对于最大训练长度的平均计算成本。

在堆叠交叉注意力 (Stack Cross-Attention, STCA) 架构下, 这种随机训练策略带来两个关键挑战:

- (1) **批次级负载均衡**: 变长序列会造成 GPU 工作负载不均衡, 因为批处理时间由最长序列决定, 从而削弱潜在的 FLOPs 节省。
- (2) **子序列选择**: 有效的选择策略必须在不降低模型准确率的前提下尽量缩短训练序列长度(即最小化 SS)。

3.3.1 子序列选择策略. 子序列选择过程包括两个步骤: (1) 采样一个随机训练长度 L_{train} ; (2) 选择完整历史中要保留的元素。

随机长度采样. 我们在归一化比率上使用 U 形 Beta 分布采样随机训练长度 $L_{\text{train}} \in [L_{\text{train}}^{\min}, L_{\text{train}}^{\max}]$, 使训练课程在大量极短窗口与偶尔出现的长窗口之间混合, 同时将平均长度保持在 $L_{\text{train}}^{\text{avg}}$ (从而控制 SS 与 ρ_{extra})。这种长度采样可视为一种数据课程; 端到端目标仍是式 (10) 中相同的 BCE 损失。确切的采样公式、期望约束与硬件对齐细节见附录 A.4。

元素选择策略. 给定 L_{train} , 我们从用户的完整历史 (推理时截断至 L_{infer}) 中选择相应数量的条目。实验结果表明, 保留最近的 L_{train} 次交互, 即时间后缀, 始终能获得最优准确率。

3.3.2 批次级负载均衡. 变长的 L_{train} 可能造成批次级工作负载不均衡 (单步耗时由最长序列主导)。因此, 我们采用批次级负载均衡算子, 使总 token 预算接近 $B \cdot L_{\text{train}}^{\text{avg}}$ (批大小为 B), 并压紧序列以减少填充, 同时保留随机长度课程。我们使用基于索引的目标注意力核函数实现由此产生的不规则计算, 以避免填充开销。完整的算子细节与不规则注意力实现见附录 A.4。

4 实验

4.1 Douyin 离线对比

4.1.1 实验设置. 我们在 *Douyin* 离线数据集上针对三个目标评估序列编码器——**finish** (完播)、**skip** (快速跳过) 和 **head** (创作者主页点击)——报告 AUC (越高越好) 和 NLL (越低越好)。为使比较更为保守,

表 1: 在 Douyin 上的离线结果。所有数值的单位均为 %。

模型	完播		跳过		头部	
	$\Delta AUC \uparrow$	$\Delta NLL \downarrow$	$\Delta AUC \uparrow$	$\Delta NLL \downarrow$	$\Delta AUC \uparrow$	$\Delta NLL \downarrow$
基线	0.00	0.00	0.00	0.00	0.00	0.00
DIN	+0.19	-0.17	+0.23	-0.18	+0.19	-0.21
Trans	+0.25	-0.46	+0.27	-0.36	+0.38	-0.27
HSTU	+0.31	-0.86	+0.52	-0.62	+0.36	-0.43
本文方法	+0.49	-1.16	+0.71	-1.14	+0.39	-1.41

所有基线均加入了 $TWIN(10k)$ [31] (一种基于长度为 10k 的行为检索构建的检索式模块), 而我们的方法移除了 $TWIN(10k)$, 完全依赖端到端长历史建模。该设置刻意保持保守并有利于基线, 因为基线获得了额外的检索信号。我们的论点是, STCA+RLB+Ext 能够以相当的端到端成本替代这一重型检索模块, 同时保留长历史上的完全可微性。¹ 重要的是, 我们在计算量大致匹配的条件下进行比较: 各方法的单样本序列 FLOPs 和单步耗时均对齐。对于二次成本的编码器 (Transformer、HSTU), 我们减小其深度/宽度, 使其总计算量与 STCA 相当, 从而确保公平比较。

我们将单层目标注意力、DIN、Transformer (自注意力) 和 HSTU 与我们的 **STCA** (堆叠式目标 $\rightarrow$ 历史交叉注意力) 进行比较; 后者结合了 **RLB** 和 **稀疏训练/稠密推理** (“Ext”)。表格单元报告相对于生产基线 $RankMixer + Single-layer Target Attention + TWIN(10k)$ 的百分比 (%) 变化; ΔAUC 为正、 ΔNLL 为负表示效果更好。

4.1.2 结果与讨论. 在表 1 中, 尽管未使用任何 $TWIN(10k)$, **STCA+RLB+Ext** 仍在所有任务上取得了最显著的提升: 完播为 **+0.49/-1.16**, 跳过为 **+0.71/-1.14**, 头部为 **+0.39/-1.41**。在计算量匹配的设置下, 各基线也有所改善: DIN 最高达到 **+0.23/-0.21**, Transformer 最高达到 **+0.38/-0.46**, HSTU 最高达到 **+0.52/-0.86**; 但本文方法始终带来最大增益, 尤其是在 NLL 上 (例如头部任务为 **-1.41**)。

¹所有模型均采用相同的非序列特征、优化器和数据划分; 差异仅在于序列编码器及是否使用 $TWIN(10k)$ 。

这些增益与我们的设计一致：检索特征会预先筛选信息，并损失部分信息和端到端梯度；**STCA** 以每个目标 $O(L)$ 的成本在完整历史上执行精确的 **softmax** 注意力，**RLB** 消除了跨目标的冗余用户编码，而**稀疏训练/稠密推理**通过暴露经过校准的长上下文尾部，使模型无需全长训练即可在数千 token 上进行推理。

结论。在非序列特征相同且计算量匹配的条件下，**STCA+RLB+Ext** 同时改善排序与校准，并简化了整体技术栈（无需 **TWIN(10k)**），为在 Douyin 上实现准确且可部署的长序列建模提供了一条实用途径。

4.2 消融实验：准确率与模型复杂度

设置与指标。我们评估端到端系统栈 **STCA** $\rightarrow$ **RankMixer**，其中使用 **RLB** ($m=8$) 和单查询交叉注意力（第 3 节）。除非另有说明，训练和评估均使用 $L=512$ ；我们报告相对于一个强 **RankMixer** 单独基线的 *finish AUC* 提升（%），该基线采用相同的非序列特征和优化方式训练。

增益来源。表 2 中的消融实验表明，在序列路径上增加逐 token FFN，并将 STCA 从 $2L \rightarrow 4L$ 加深，带来了最大的单项提升（+0.18%）。将 FFN 升级为 *SwiGLU* 又带来 +0.11% 的提升。扩大稀疏 ID 嵌入（+0.08%）和引入时间差侧信息（+0.08%）也都作出了显著贡献。增加注意力头数 $8 \rightarrow 16$ 带来了幅度不大但为正的增益（+0.05%）。最后，查询融合机制（式 7）通过将较低层摘要重新注入较高层以进行目标条件推理，又增加了 +0.06%。

计算量-质量扩展。图 3 在深度/宽度匹配的条件下将计算量 (FLOPs) 与质量 (NLL) 联系起来。有两点尤为突出：

- **关于 L 的线性增长与二次增长。**STCA 的序列侧 FLOPs 呈线性增长，而 Transformer 呈二次增长。从 $L=500 \rightarrow 10k$ 开始，STCA 增加 $1.06 \rightarrow 21.06$

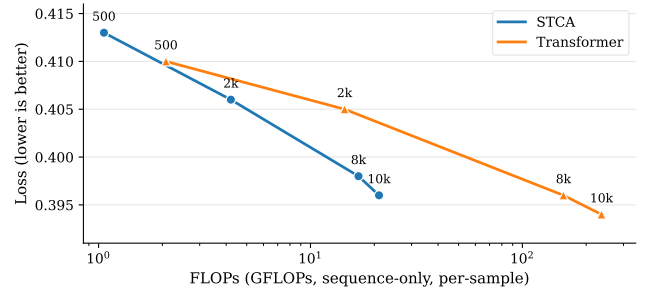


图 3: **STCA** 与 **Transformer** 的计算量-质量扩展关系。横轴：每个样本仅序列部分的前向 FLOPs（对数尺度）。纵轴：NLL（越低越好）。标记表示 $L \in \{500, 2k, 8k, 10k\}$ 。二者均使用 4 层，且 $d=256$ 、 $h=8$ 、 $r=4$ 。

GFLOPs ($\sim 19.9\times$)，而 Transformer 增加 $2.08 \rightarrow 236.26$ GFLOPs ($\sim 113.6\times$)。

- **长 L 下更优的前沿。**在相近的 NLL (≈ 0.396) 下，STCA 在 $L=10k$ 时使用 21.06 GFLOPs，而 Transformer 需要 $L=8k$ 和 156.24 GFLOPs (高 $\sim 7.4\times$)。

这些结果与我们的更广泛发现一致：**STCA** 移除历史自注意力，将计算集中于目标 $\leftrightarrow$ 历史相关性（近线性扩展）；**RLB** 摊销用户路径成本（并可跨请求/会话扩展）；**稀疏训练/稠密推理**在 $\sim 2k$ 上训练，但服务时可扩展至 10k。三者结合提供了一条实用的扩展路径：STCA 能在相近或更低的 FLOPs 下支持更长上下文，同时保持或提升排序质量。

4.3 请求级批处理 (RLB)

设置。RLB 在请求边界实现以用户为中心的批处理：对于每个包含一份用户历史和多个目标的请求，我们只传输并编码一次共享的用户/上下文载荷，将其复用于所有目标，并在同步前于请求级聚合梯度。除非另有说明，模型栈为启用 RLB (第 3 节) 的 **STCA** $\rightarrow$ **RankMixer** 并采用与逐样本基线相同的数据、优化器和批大小进行训练。报告的带宽数据包括所有非序列特征（画像/上下文、内容、创作者等），即端到端载荷。

表 2: 在 512 个 token 下对 4L 复杂 STCA 的消融实验。

组件	Δ AUC	说明
扩大稀疏 ID 嵌入: 128 $\rightarrow$ 320	+0.08%	Video-ID 嵌入宽度。采用更小的初始化范围和学习率以避免不稳定。
增加序列侧 FFN; 深度 2 $\rightarrow$ 4	+0.18%	历史路径上的逐 token FFN; 随后将 STCA 深度从 2L 加倍至 4L。
FFN $\rightarrow$ SwiGLU	+0.11%	将 FFN 升级为 SwiGLU; 隐藏层加宽 $\times 2$ 。
注意力头数: 8 $\rightarrow$ 16	+0.05%	更多注意力头以较小成本提升目标条件下的选择性。
查询融合	+0.06%	在第 $i+1$ 层, 拼接 $[\mathbf{o}^{(1)} \dots \mathbf{o}^{(i)}, \mathbf{x}_t]$, 随后应用 SwiGLU (式 7)。
时间差侧信息	+0.08%	增加逐 token 特征: 请求时间减去条目时间戳 (近期性先验)。

带宽占用。RLB 通过在多个目标之间摊销 $O(L)$ 用户/历史载荷, 显著减少模块间流量: 在历史长度为 $L=512$ 时, 实测降幅为 **77%**; 在 $L=2k$ 时为 **84%** (二者均包含所有现有特征)。在更长 L 下节省更多, 直接源于避免为每个目标重复传输相同历史。

吞吐量与可扩展性。相较于逐样本基线 ($1\times$), RLB 将端到端训练吞吐量提升了 **2.2 $\times$** 。进一步进行内核优化后——包括针对重排后单查询注意力的专用批量矩阵乘法、高吞吐量 *SwiGLU* 和优化的 Layer-Norm——加速比达到 **5.1 $\times$** 。在相同基础设施下, 通过摊销每目标激活和载荷, 最大可训练序列长度也提高了约 **8 $\times$** (在长度 L 时受内存/IO 限制的工作负载, 在 $\approx 8L$ 时变得可训练)。

参数服务器与模块间成本。在同步和特征服务边界, RLB 进一步降低了开销: 我们观察到训练期间参数服务器 (PS) 的 CPU 使用率降低了 **50%**, 数据 $\leftrightarrow$ 训练通信带宽降低了 **50%**; 训练侧 PS 的 CPU 负载同样降低了 **50%**。

讨论。这些增益与 STCA 的架构效率相辅相成。STCA 通过单查询交叉注意力 ($O(L)$) 消除了对历史长度的二次依赖, 而 RLB 则消除了同一用户/历史在不同目标之间的冗余传输和重复编码。二者结合可提高每秒处理样本数、降低 PS 争用, 并大幅增加可行的序列长度, 使长上下文模型能够在固定延迟和硬件预算内运行。

表 3: 最大训练长度的影响。

指标	$L_{\text{train}}^{\max} = 2k$	$L_{\text{train}}^{\max} = 4k$	$L_{\text{train}}^{\max} = 10k$
完播 AUC 提升	+0.03%	+0.09%	+0.21%

表 4: 平均训练长度的影响。

指标	$L_{\text{train}}^{\text{avg}} = 1.0k$	$L_{\text{train}}^{\text{avg}} = 2.0k$	$L_{\text{train}}^{\text{avg}} = 2.5k$
完播 AUC 提升	+0.09%	+0.21%	+0.22%
序列稀疏度	10.0%	20.0%	25.0%

4.4 外推: 稀疏训练, 稠密推理

实验设置。我们使用带单查询优化的 STCA 编码器评估所提出的外推框架。用户 token $\mathbf{z}$ 被送入采用 RLB 训练 ($m=8$) 的 *RankMixer* (第 3 节)。训练期间, 历史长度 L_{train} 按照第 3.3.1 节随机化。所有结果均报告相对于固定 2k-token 基线的完播 AUC 提升。除非另有说明, **推理使用 $L_{\text{infer}} = 10k$** 。

最大长度的影响 (表 3)。将 $L_{\text{train}}^{\max}$ 从 2k 逐步增至 10k, AUC 提升也相应改善, 从微小的 +0.03% 增至更显著的 +0.21%。这些结果证实, 为确保对长上下文的稳健泛化, 训练方案必须让模型接触与推理时序列长度接近或近似的序列。

序列稀疏度优化。表 4 展示了效率与准确率之间的权衡。将 $L_{\text{train}}^{\text{avg}}$ 从 1.0k 增至 2.5k, 可使 AUC 提升从 +0.09% 增至 +0.22%, 同时序列稀疏度 (SS)

表 5: Beta 分布形状参数分析。

指标	$\alpha = 0.02$	$\alpha = 0.5$	$\alpha = 10$
完播 AUC 提升	+0.21%	+0.11%	+0.08%
形状	U 形	递减	偏斜

从 10% 增至 25%。超过 $\bar{L} = 2.0k$ 后收益递减, 说明 $SS \approx 20\%$ 实现了最优平衡。与 HSTU 中的 SL ($SS=57.6\%$) [37] 相比, 我们获得了更高的计算效率。

子序列选择策略验证。· 保留最近的交互 (贪心策略) 可获得 +0.21% 的 AUC 提升, 而随机采样没有带来增益, 这有力地证明了时间局部性的重要性。

Beta 分布形状分析。· 表 5 验证了我们的分布设计: 与递减分布 (+0.11%) 和偏斜分布 (+0.08%) 相比, U 形分布 ($\alpha = 0.02$) 取得了更优的性能 (+0.21%), 证实双峰采样优化了训练课程。

效率—准确率权衡。· 我们的方法在 10k 推理下实现了 +0.23% 的离线 AUC 提升, 以三分之一的计算成本保留了完整 10k 训练增益 (+0.30%) 的 $\sim 80\%$ 。在线 A/B 测试证实了其生产可行性 (+0.17% 完播 AUC)。

讨论。· 结果全面验证了我们的方法: (1) 使用 U 形 Beta 分布的随机训练能够实现有效外推; (2) 时间后缀选择保留了序列模式; (3) 负载均衡保证了训练效率。在我们的默认设置 $L_{\text{train}}^{\text{avg}} = 2k$ 、 $L_{\text{train}}^{\text{max}} = 10k$ 和 $L_{\text{infer}} = 10k$ ($\rho_{\text{extra}} = 5$) 下。与 HSTU 的 SL ($SS=57.6\%$) 相比, 我们在保持准确率的同时将 SS 降至 20%, 展现出更优的计算效率。

4.5 在线 A/B 测试结果

实验设置。· 我们在 *Douyin* 和 *Douyin Lite* 上部署了一个月的 **STCA + RLB + Extrapolation** (第 3 节), 使用我们的单查询目标 $\rightarrow$ 历史编码器替代加入 TWIN(10k) 的检索特征, 同时保持其他所有组件不变。我们报告相对于对照组在 **30 日活跃度**、应

用停留时长、完播、评论和点赞方面的百分比提升, 包括总体结果以及按用户活跃度划分的结果 (表 6)。

推理成本。· 在我们的内部核算中, 移除 TWIN 会使 GPU 成本增加 +33%, 但 CPU 成本降低 -16% , 估计总成本净变化约为 +17%; 考虑到在线增益, 我们认为这一变化可以接受。

实验发现。· 我们的方法在两个产品及所有用户分群上均取得了一致且可观的在线增益。总体而言, **完播和应用停留时长**同步改善, 交互信号 (**评论、点赞**) 也显著上升。对于低/中活跃度用户, 增益最为突出, 这表明面对稀疏或嘈杂的近期行为时, 个性化能力得到了改善; 与此同时, **30 日活跃度**也呈现幅度较小但稳定的提升。这些改善源于: (i) **STCA** 将计算集中于长上下文中的精确目标 $\rightarrow$ 历史交互; (ii) **RLB** 摊销用户编码成本, 使服务成本保持在预算内; (iii) **稀疏训练/稠密推理**在训练时让模型接触经过校准的长序列尾部。三者共同使端到端长历史建模既准确, 又能进行大规模部署。

5 相关工作与讨论

5.1 用户行为序列建模

早期系统采用 item-to-item CF 和马尔可夫转移等非深度方法对序列建模 [18, 29]。随后, 深度学习带来了基于会话的 RNN 和工业级两阶段系统栈 (例如 YouTube) [5, 10]。注意力/Transformer 模型逐渐占据主导地位: DIN/DIEN 以候选为条件处理历史, 从而突出目标感知选择 [39, 40]; SASRec/BERT4Rec 则提供强大的自注意力基线, 通过双向或单向注意力建模时间依赖关系 [13, 33]; BST 进一步展示了在工业信息流排序场景中的在线部署 [3]。与此同时, 多兴趣建模显式捕获用户的多样化意图, 已广泛应用于大规模平台 [16]。

近期的生产工作强调长历史和系统兼容性: 用户表征方法 (例如 PinnerFormer) 将长期行为压缩为凝练记忆, 而实时/批处理融合 (例如 TransAct) 则将流式信号与预计算特征合并, 以实现延迟感知的服务 [24, 34, 35]。为进一步扩展, 两阶段范式会在精细建

表 6: 相对于对照组, 在抖音和抖音极速版上进行为期一个月的在线 A/B 测试提升 (%)。

用户分层	Douyin					Douyin Lite				
	30 日活跃	停留时长	完播	评论	点赞	30 日活跃	停留时长	完播	评论	点赞
低	0.3659%	2.0070%	5.4987%	2.6848%	2.4367%	0.3575%	1.3888%	6.2808%	6.3385%	3.1623%
中	0.3788%	1.7065%	5.2062%	0.7028%	2.3779%	0.2832%	1.3172%	5.9688%	5.7934%	5.6372%
高	0.1396%	1.1262%	3.7973%	1.4012%	2.1455%	0.1604%	0.9872%	4.9922%	2.8496%	3.1752%
所有用户	0.1161%	0.9266%	3.3454%	1.5678%	1.8282%	0.1281%	0.8467%	4.2275%	2.6167%	2.3828%

模之前检索与目标相关的片段(SIM、UBR4CTR) [25, 27], 或者设计稀疏记忆/层次结构(SAMN、TWIN/TWIN-V2), 以限制处理超长日志的成本 [2, 17, 31]。这些设计通常通过截断、检索或摘要来提高效率, 但会牺牲完整序列上的精确端到端梯度。与这些方法相比, 我们的 *STCA* 在完整序列上端到端执行单查询目标 $\rightarrow$ 历史交叉注意力, 实现关于 L 的线性复杂度, 并避免检索/截断。这为在生产环境中使用超长历史提供了一条实用途径, 同时保持所有已观测交互的可微性。

5.2 训练样本组织

经典训练形式包括逐点/成对学习 (例如 BPR) [28]、大批量 CTR 流水线 (DLRM、生产系统) [22, 23], 以及序列的会话并行批处理 [10]。在工业规模下, 训练通常受 I/O/内存而非 FLOP 限制 [21], 这促使人们采用截断或检索优先的构造方式 [25, 27], 并针对超长上下文开展系统工程优化 [2, 17, 31]。现代系统栈 (例如 TorchRec) 提供锯齿张量和分片, 以更充分地利用硬件并管理稀疏性 [12]; 生产系统还会混合实时/批处理信号, 以平衡新鲜度与稳定性 [24, 34]。在这一背景下, 我们的请求级批处理 (*RLB*) 在用户/请求粒度上重组数据: 每个请求仅计算一次用户/历史编码, 并在 m 个目标之间复用, 将每个目标的用户路径成本从 $O(L)$ 降至 $\approx O(L/m)$, 同时保持经验风险目标无偏。与截断/检索不同, *RLB* 保留完整历史, 并摊销数据移动和重复计算开销, 直接解决大规模场景下占主导地位的 I/O 与激活瓶颈 [21]。它与检索、内核级加速及分片 (例如 [12] 中的锯齿张量) 相互正交, 并

与生产环境中的请求级分组相契合, 在不改变损失函数的情况下改善带宽、峰值内存和内核效率。

5.3 长度外推

LLM 表明, 通过位置设计 (ALiBi、RoPE) 以及可推广至训练窗口之外的记忆/注意力模式 (Transformer-XL、Longformer、BigBird), “稀疏训练, 稠密推理”是可行的 [1, 6, 26, 32, 36]。我们将这一思想应用于推荐系统: 其一, 以精确的单查询交叉注意力 (*STCA*) 替代二次复杂度的历史自注意力; 其二, 通过 *RLB* 重组训练, 以分摊用户编码成本。与检索流水线 [2, 25, 27, 31] 不同, 我们保留了完整历史上的端到端梯度; 与核方法/稀疏方法 [1, 36] 不同, 我们直接匹配目标与历史之间的交互, 同时满足生产约束 [12, 21, 24, 34]。

具体而言, 我们在训练期间采用随机长度采样, 使平均序列长度保持较短, 同时暴露经过校准的较长上下文尾部; 推理时则使用长得多的历史进行服务。该方案利用 *STCA* 关于 L 线性的计算量与 *RLB* 的分摊机制, 使外推无需截断或替代记忆即可在生产中切实落地。

6 结论

我们提出了一套用于长序列推荐的端到端方案, 在架构、系统与训练三个层面同时具备高效率。在架构层面, *STCA* 用单查询目标 $\rightarrow$ 历史交叉注意力替代历史自注意力, 使复杂度随序列长度线性 ($O(L)$) 增长。在系统层面, *RLB* 在请求级复用用户侧编码, 消除冗余传输与计算。在训练层面, “稀疏训练/稠密推理”

方案以适中的训练成本实现长历史上的稠密推理。离线与在线实验表明, 随着序列长度和序列模块容量增长, 模型可获得稳定、显著的增益以及类似缩放定律的改善。主要发现包括: 单查询注意力足以用于短视频排序, 同时保持 $O(L)$ 成本; 采用较小 α 的 Beta 随机长度采样, 能以约三分之一的训练成本获得 10k 窗口约 80% 的收益; 参数预算最好通过 SwiGLU、跨层查询融合和时间差特征投入序列路径, 而且上下文越长, 这些设计的影响越大; 系统层面的优化手段对可部署性至关重要——在 ($L=2k$) 下, RLB 将端到端带宽最多降低约 84%, 将 PS CPU 使用率减半, 实现约 $2.2\times$ 的吞吐, 并将最大可训练序列长度扩大约 $8\times$ 。在生产环境中, 我们观察到每位用户平均聚类内容类别数增加 1.6%, 表明多样性略有提升。

实践中, 一个效果良好的工作点是: 将 4 层 STCA (采用 SwiGLU 与查询融合) 和时间差特征、RLB、随机长度训练 (较小的 α , 训练平均长度 $\approx 2k$) 结合, 并将推理长度设为 10k (外推比率为 $5\times$)。该配置可随序列长度与序列模块容量的增加持续获得单调改善, 同时将训练与服务成本控制在预算内, 为准确的长序列推荐提供一条生产就绪的路径。

References

- [1] Iz Beltagy, Matthew E. Peters, and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150*
- [2] Jianxin Chang, Chenbin Zhang, Zhiyi Fu, Xiaoxue Zang, Lin Guan, Jing Lu, Yiqun Hui, Dewei Leng, Yanan Niu, Yang Song, and Kun Gai. 2023. TWIN: TTwo-stage Interest Network for Life-long User Behavior Modeling in CTR Prediction at Kuaishou. In *KDD*.
- [3] Qiwei Chen, Huan Zhao, Wei Li, Pipei Huang, and Wenwu Ou. 2019. Behavior Sequence Transformer for E-commerce Recommendation in Alibaba. *arXiv:1905.06874*
- [4] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishikesh Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Isipir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. In *DLRS@RecSys*.
- [5] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *RecSys*.
- [6] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G. Carbonell, Quoc Viet Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive Language Models beyond a Fixed-Length Context. In *ACL*.
- [7] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *NeurIPS*.
- [8] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *WWW*.
- [9] Zhicheng He, Weiwen Liu, Wei Guo, Jiarui Qin, Yingxue Zhang, Yaochen Hu, and Ruiming Tang. 2023. A Survey on User Behavior Modeling in Recommender Systems. In *IJCAI*.
- [10] Balázs Hidasi, Alexandros Karatzoglou, Linas Baltrunas, and Domonkos Tikk. 2016. Session-based Recommendations with Recurrent Neural Networks. In *ICLR*.
- [11] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training Compute-Optimal Large Language Models. *arXiv:2203.15556*
- [12] Dmytro Ivchenko, Dennis Van Der Staay, Colin Taylor, Xing Liu, Will Feng, Rahul Kindi, Anirudh Sudarshan, and Shahin Sefati. 2022. TorchRec: a PyTorch Domain Library for Recommendation Systems. In *RecSys*.
- [13] Wang-Cheng Kang and Julian J. McAuley. 2018. Self-Attentive Sequential Recommendation. In *ICDM*.
- [14] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. *arXiv:2001.08361*
- [15] Barrie Kersbergen, Olivier Sprangers, and Sebastian Schelter. 2022. Serenade - Low-Latency Session-Based Recommendation in e-Commerce at Scale. In *SIGMOD*.
- [16] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Huan Zhao, Pipei Huang, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. 2019. Multi-Interest Network with Dynamic Routing for Recommendation at Tmall. In *CIKM*.
- [17] Qianying Lin, Wen-Ji Zhou, Yanshi Wang, Qing Da, Qing-Guo Chen, and Bing Wang. 2022. Sparse Attentive Memory Network for Click-through Rate Prediction with Long Sequences. In *CIKM*.
- [18] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Comput.* 7, 1 (2003), 76–80.
- [19] Liang Luo, Buyun Zhang, Michael Tsang, Yinbin Ma, Ching-Hsiang Chu, Yuxin Chen, Shen Li, Yuchen Hao, Yanli Zhao, Guna Lakshminarayanan, Ellie Wen, Jongsoo Park, Dheevatsa Mudigere, and Maxim Naumov. 2024. Disaggregated Multi-Tower: Topology-aware Modeling Technique for Efficient Large Scale Recommendation. In *MLSys*.
- [20] Benjamin M. Marlin, Richard S. Zemel, Sam T. Roweis, and Malcolm Slaney. 2007. Collaborative Filtering and the Missing at Random Assumption. In *UAI*.

- [21] Dheevatsa Mudigere, Yuchen Hao, Jianyu Huang, Zhihao Jia, Andrew Tulloch, Srinivas Sridharan, Xing Liu, Mustafa Ozdal, Jade Nie, Jongsoo Park, Liang Luo, Jie Amy Yang, Leon Gao, Dmytro Ivchenko, Aarti Basant, Yuxi Hu, Jiyang Yang, Ehsan K. Ardestani, Xiaodong Wang, Rakesh Komuravelli, Ching-Hsiang Chu, Serhat Yilmaz, Huayu Li, Jiyuan Qian, Zhuobo Feng, Yinbin Ma, Junjie Yang, Ellie Wen, Hong Li, Lin Yang, Chonglin Sun, Whitney Zhao, Dmitry Melts, Krishna Dhulipala, K. R. Kishore, Tyler Graf, Assaf Eisenman, Kiran Kumar Matam, Adi Gangidi, Guoqiang Jerry Chen, Manoj Krishnan, Avinash Nayak, Krishnakumar Nair, Bharath Muthiah, Mahmoud khorashadi, Pallab Bhattacharya, Petr Lapukhov, Maxim Naumov, Ajit Mathews, Lin Qiao, Mikhail Smelyanskiy, Bill Jia, and Vijay Rao. 2022. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *ISCA*.
- [22] Maxim Naumov, John Kim, Dheevatsa Mudigere, Srinivas Sridharan, Xiaodong Wang, Whitney Zhao, Serhat Yilmaz, Changkyu Kim, Hector Yuen, Mustafa Ozdal, Krishnakumar Nair, Isabel Gao, Bor-Yiing Su, Jiyang Yang, and Mikhail Smelyanskiy. 2020. Deep Learning Training in Facebook Data Centers: Design of Scale-up and Scale-out Systems. arXiv:2003.09518
- [23] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azcolini, Dmytro Dzhulgakov, Andrey Mallevech, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, and Misha Smelyanskiy. 2019. Deep Learning Recommendation Model for Personalization and Recommendation Systems. arXiv:1906.00091
- [24] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. PinnerFormer: Sequence Modeling for User Representation at Pinterest. In *KDD*.
- [25] Qi Pi, Guorui Zhou, Yujing Zhang, Zhe Wang, Lejian Ren, Ying Fan, Xiaoqiang Zhu, and Kun Gai. 2020. Search-based User Interest Modeling with Lifelong Sequential Behavior Data for Click-Through Rate Prediction. In *CIKM*.
- [26] Ofir Press, Noah A. Smith, and Mike Lewis. 2022. Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation. In *ICLR*.
- [27] Jiarui Qin, Weinan Zhang, Xin Wu, Jiarui Jin, Yuchen Fang, and Yong Yu. 2020. User Behavior Retrieval for Click-Through Rate Prediction. In *SIGIR*.
- [28] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian Personalized Ranking from Implicit Feedback. In *UAI*.
- [29] Steffen Rendle, Christoph Freudenthaler, and Lars Schmidt-Thieme. 2010. Factorizing personalized Markov chains for next-basket recommendation. In *WWW*.
- [30] Tobias Schnabel, Adith Swaminathan, Ashudeep Singh, Navin Chandak, and Thorsten Joachims. 2016. Recommendations as Treatments: Debiasing Learning and Evaluation. In *ICML*.
- [31] Zihua Si, Lin Guan, Zhongxiang Sun, Xiaoxue Zang, Jing Lu, Yiqun Hui, Xingchao Cao, Zeyu Yang, Yichen Zheng, Dewei Leng, Kai Zheng, Chenbin Zhang, Yanan Niu, Yang Song, and Kun Gai. 2024. TWIN V2: Scaling Ultra-Long User Behavior Sequence Modeling for Enhanced CTR Prediction at Kuaishou. In *CIKM*.
- [32] Jianlin Su, Murtadha H. M. Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. RoFormer: Enhanced transformer with Rotary Position Embedding. *Neurocomputing* 568 (2024), 127063.
- [33] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *CIKM*, Wenwu Zhu, Dacheng Tao, Xueqi Cheng, Peng Cui, Elke A. Rundensteiner, David Carmel, Qi He, and Jeffrey Xu Yu (Eds.).
- [34] Xue Xia, Pong Eksombatchai, Nikil Pancha, Dhruvil Deven Badani, Po-Wei Wang, Neng Gu, Saurabh Vishwas Joshi, Nazanin Farahpour, Zhiyuan Zhang, and Andrew Zhai. 2023. TransAct: Transformer-based Realtime User Action Model for Recommendation at Pinterest. In *KDD*.
- [35] Xue Xia, Saurabh Vishwas Joshi, Kousik Rajesh, Kangnan Li, Yangyi Lu, Nikil Pancha, Dhruvil Deven Badani, Jiajing Xu, and Pong Eksombatchai. 2025. TransAct V2: Lifelong User Action Sequence Modeling on Pinterest Recommendation. arXiv:2506.02267
- [36] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for Longer Sequences. In *NeurIPS*.
- [37] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Jiayuan He, Yinghai Lu, and Yu Shi. 2024. Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations. In *ICML*.
- [38] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep Learning Based Recommender System: A Survey and New Perspectives. *ACM Comput. Surv.* 52, 1 (2019), 5:1–5:38.
- [39] Guorui Zhou, Na Mou, Ying Fan, Qi Pi, Weijie Bian, Chang Zhou, Xiaoqiang Zhu, and Kun Gai. 2019. Deep Interest Evolution Network for Click-Through Rate Prediction. In *AAAI*. 5941–5948.
- [40] Guorui Zhou, Xiaoqiang Zhu, Chengru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction. In *KDD*.
- [41] Jie Zhu, Zhifang Fan, Xiaoxie Zhu, Yuchen Jiang, Hangyu Wang, Xintian Han, Haoran Ding, Xinmin Wang, Wenlin Zhao, Zhen Gong, Huizhi Yang, Zheng Chai, Zhe Chen, Yuchao Zheng, Qiwei Chen, Feng Zhang, Xun Zhou, Peng Xu, Xiao Yang, Di Wu, and Zuotao Liu. 2025. RankMixer: Scaling Up Ranking Models in Industrial Recommenders. arXiv:2507.15551

A 更多实现与系统细节

本附录提供因篇幅所限而未在正文中展开的其他实现与系统细节。内容组织如下：(i) 训练流水线与模块化集成；(ii) STCA 实现细节与注意力效率；(iii) 作为样本组织原语的请求级批处理（RLB）；以及 (iv) 完整的外推流程，包括 Beta 采样与批次级负载均衡。

A.1 训练流水线与模块化集成

直接在长上下文上训练可能不稳定（例如 $L=2048$ ）。因此，我们采用一个简单的课程：(i) 先在 $L=512$ 上预训练，以建立稳健的 token 级过滤器和注意力模式；(ii) 随后在 $L=2048$ 上继续训练。在架构迭代期间，我们在 $L=512$ 上制作原型，以实现更快收敛并减少资源使用。为了集成到更大的生产模型栈中，我们首先将序列子网络训练至收敛，再将其参数加载到复合模型中，最后进行联合微调。当模型栈的其余部分已经很强时，这种分阶段方式可以缓解序列路径上的梯度消失问题。

A.2 STCA 细节：稳健性与注意力效率

A.2.1 STCA 的泛化能力与稳健性. STCA 能够在不同序列长度上良好泛化，因为每个历史 token 都在以目标为条件的情况下独立处理，使该架构天然不依赖长度；堆叠机制使信息聚合能够随 L 增加而平稳扩展。通过在每一层都借助目标过滤历史，STCA 对真实日志中常见的无关或有噪声行为不那么敏感。这提高了模型面对变长序列和异质用户模式时的稳健性，而这对于大规模工业部署至关重要。

A.2.2 单查询注意力重排与 FLOPs 分析. 当每层恰好只有一个查询时，令 $X \in \mathbb{R}^{L \times d}$ 、 $q \in \mathbb{R}^{1 \times d}$ 和 $d_h = d/h$ 。标准交叉注意力形式为

$$\text{Attn}(q, X) = \text{softmax}\left(\frac{(qW_Q)(XW_K)^T}{\sqrt{d_h}}\right) \cdot (XW_V), \quad (14)$$

该形式会对全部 L 个 token 各投影两次，并物化长度为 L 的张量 XW_K 与 XW_V 。

重排。· 我们可以重排计算，以消除长度为 L 的投影：

$$u = (qW_Q)W_K^T \in \mathbb{R}^{1 \times d}, \quad \alpha = \text{softmax}\left(\frac{uX^T}{\sqrt{d_h}}\right) \in \mathbb{R}^{1 \times L},$$

然后计算

$$o = (\alpha X)W_V \in \mathbb{R}^{1 \times d_h}, \quad W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}.$$

由此得到式 (11) 所概括的等价实现：

$$\text{Attn}(q, X) = \left(\text{softmax}\left(\frac{(qW_Q)W_K^T X^T}{\sqrt{d_h}}\right)X\right)W_V = (\alpha X)W_V.$$

FLOPs 与中间量. · 对于每个注意力头，重排后的路径成本为 $O(dd_h) + O(Ld) + O(dd_h)$ ；在 h 个头合计为 $O(Ldh + d^2)$ ，同时避免任何 $L \times d_h$ 中间量。相比之下，朴素路径在形成 (XW_K, XW_V) 时需要 $\approx 4Ldd_h$ FLOPs，并为每个头物化两个 $L \times d_h$ 张量。重排后的路径用一次加权归约 αX 取代它们，其成本为 $2Ld$ FLOPs，且不产生 $L \times d_h$ 中间量。因此，与长度相关的 FLOPs 缩减了 $\approx 2d_h = 2d/h$ 倍。

示例. · 当 $d=256$ 且 $h=8$ ($d_h=32$) 时，与长度相关的 FLOPs 降幅为 $\sim 2d_h = \sim 64 \times$ 。

A.2.3 与注意力优化器和融合核函数的关系. GQA/MQA

通过减少不同 K/V 投影的数量来节省内存/带宽，但其在长度为 L 的序列上进行打分的成本仍为 $O(L^2d)$ 。I/O 高效核函数（如 FlashAttention）可减少内存流量，但仍保留二次计算；线性/低秩变体则通过近似达到 $O(Ld)$ 。**STCA** 不同，它完全移除了历史内部的自交互，并以每层 $O(Ldh)$ 的成本执行精确的单查询目标 $\rightarrow$ 历史注意力（还可进一步受益于式 (11) 中的重排），而下游 RankMixer 只处理一个规模较小且与长度无关的 token 集。**RLB** 则通过将用户路径分摊到 m 个目标上对此形成补充，相当于将每个目标的 $O(L)$ 转化为 $O(L/m)$ 。这些技术与注意力头共享和融合核函数兼容，但我们的主要节省来自：(i) 在架构层面移除二次项；(ii) 在系统层面进行分摊，且无需检索或截断即可实现。

A.3 作为样本组织方式的请求级批处理 (RLB)

逐实例 (三元组) 批处理会对每个 (u, v, y) 重新编码完整历史 $\mathcal{H}$, 使数据集大小和 I/O 按 $O(mL)$ 增长。按长度填充/分桶可缓解内核分歧, 但仍会重复用户编码。历史截断/检索优先通过缩短或选择子序列来降低成本, 但会丢弃信息, 并中断完整历史上的端到端梯度。嵌入缓存/聚类以近似误差为代价压缩历史。相比之下, **请求级批处理 (RLB)** 是无损且端到端的: 它保留完整的 $\mathcal{H}$, 保持目标函数不变, 并将每个目标的用户路径复杂度从 $O(L)$ 降至约 $O(L/m)$ 。在实践中, 我们设置 $m=8$, 使用户侧带宽和编码器计算降低约 $8\times$, 提高 GPU 利用率, 并支持更长的 L 。

A.4 外推细节: Beta 采样与负载均衡

A.4.1 采用 Beta 课程的随机长度采样. 我们从 Beta 分布中采样归一化长度比例 $s \in (0, 1)$, 并将其映射为训练长度:

$$s \sim \text{Beta}(\alpha, \beta),$$

$$L_{\text{train}}^{\text{raw}} = L_{\text{train}}^{\min} + s \cdot (L_{\text{train}}^{\max} - L_{\text{train}}^{\min}). \quad (15)$$

我们采用 Beta 分布, 是因为它能够实现实验上更优的 U 形 (双峰) 分布, 且只有两个参数, 便于调优。这种采样方式可视为一种数据课程; 端到端目标仍是式 (10) 中相同的 BCE 损失。

硬件对齐与离散化. 为满足硬件加速要求 (例如张量核对齐), 我们将 $L_{\text{train}}^{\text{raw}}$ 舍入到最接近的 8 的倍数, 得到 L_{train} 。

期望约束. 给定目标平均训练长度 $L_{\text{train}}^{\text{avg}}$, 我们施加约束

$$\mathbb{E}[L_{\text{train}}^{\text{raw}}] = L_{\text{train}}^{\min} + (L_{\text{train}}^{\max} - L_{\text{train}}^{\min}) \cdot \frac{\alpha}{\alpha + \beta} = L_{\text{train}}^{\text{avg}},$$

由此可得

$$\beta = \alpha \cdot \frac{L_{\text{train}}^{\max} - L_{\text{train}}^{\text{avg}}}{L_{\text{train}}^{\text{avg}} - L_{\text{train}}^{\min}}. \quad (16)$$

因此, s 采样自

$$s \sim \text{Beta}\left(\alpha, \alpha \cdot \frac{L_{\text{train}}^{\max} - L_{\text{train}}^{\text{avg}}}{L_{\text{train}}^{\text{avg}} - L_{\text{train}}^{\min}}\right).$$

平均长度 $L_{\text{train}}^{\text{avg}}$ 通过 $\text{SS} = L_{\text{train}}^{\text{avg}}/L_{\text{train}}^{\max}$ 直接控制序列稀疏度, 而 $\rho_{\text{extra}} = L_{\text{infer}}/L_{\text{train}}^{\text{avg}}$ 。

A.4.2 批级负载均衡与不规则目标注意力. 变长序列可能导致工作负载不均衡, 因为单步耗时由批次中最长的序列决定。我们使用批级负载均衡算子, 每个批次的固定 token 预算为 $B \cdot L_{\text{train}}^{\text{avg}}$:

- (1) **全局长度分配:** 随机截断序列, 使每个批次的 token 总数保持接近 $B \cdot L_{\text{train}}^{\text{avg}}$, 从而在保留随机长度课程的同时改善工作负载均衡。
- (2) **序列紧凑化:** 压缩/打包 token 以减少填充; 将较长序列中的 token 重新分配给较短序列, 使批次在接近平均 token 预算的水平上处理。

为了高效处理压缩后的变长序列, 我们实现了由高吞吐量 GEMM 内核 [7] 支持的不规则目标注意力机制。我们不填充至最大长度, 而是使用一个辅助 `index` 张量标记分段边界: 键/值被展平为二维矩阵, 每个查询仅关注展平后 K/V 中与其对应的分段。